Milestone 4

Data Analysis using SQL

By-Bhavesh Gupta

Background

Amazon, a global leader in e-commerce, has achieved significant success in markets like the U.S., Europe, and Asia. In Brazil, Amazon connects small and medium businesses with millions of customers, becoming a key player. Given the similarities between Brazil and India—such as large populations and diverse consumer bases—there’s an opportunity to replicate success in India.

As a data analyst for Amazon India, your task is to analyse Amazon Brazil's data to identify trends, customer behaviours, and preferences that could be leveraged in the Indian market. This analysis will help Amazon India make informed decisions, enhance customer experience, and seize new opportunities.

Analysis 1

Q1)

```sql
-- Analysis 1
--q1 To simplify its financial reports, Amazon India needs to standardize payment val
--Round the average payment values to integer (no decimal)
--for each payment type and display the results sorted in ascending order.

SELECT
    payment_type,
    CAST(ROUND(AVG(payment_value)::numeric, 0) AS INTEGER) AS rounded_avg_payment
FROM amazon_brazil.payments
GROUP BY payment_type
ORDER BY rounded_avg_payment;
```

Data Output Messages Notifications

	payment_type character varying	rounded_avg_payment integer
1	not_defined	0
2	voucher	66
3	debit_card	143
4	boleto	145
5	credit_card	163

Q2)

```sql
--q2 To refine its payment strategy, Amazon India wants to know the distribution of orders by payment type.
--Calculate the percentage of total orders for each payment type, rounded to one decimal place,
--and display them in descending order
SELECT
    payment_type,
    ROUND((COUNT(*) * 100.0) / (SELECT COUNT(*) FROM amazon_brazil.payments), 1) AS percentage_orders
FROM amazon_brazil.payments
GROUP BY payment_type
ORDER BY percentage_orders DESC;
```

Data Output Messages Notifications

	payment_type character varying	percentage_orders numeric
1	credit_card	73.9
2	boleto	19.0
3	voucher	5.6
4	debit_card	1.5
5	not_defined	0.0

Q3)

```
--q3 Amazon India seeks to create targeted promotions for products within specific price ranges.
--Identify all products priced between 100 and 500 BRL that contain the word 'Smart' in their name.
--Display these products, sorted by price in descending order.

SELECT P.product_id ,O.price , P.product_category_name FROM amazon_brazil.product as P
join amazon_brazil.order_items as O
on P.product_id=O.product_id
where O.price between 100 and 500 and P.product_category_name Ilike'%Smart%'
order by O.price desc
```

Data Output Messages Notifications

	product_id character varying	price double precision	product_category_name character varying
1	1df1a2df8ad2b9d3aa49fd851e3145...	439.99	smart
2	7debe59b10825e89c1cbcc8b190c8...	349.99	smart
3	ca86b9fe16e12de698c955aedff0ae...	349	smart
4	ca86b9fe16e12de698c955aedff0ae...	349	smart
5	0e52955ca8143bd179b311cc454a6...	335	smart
6	7aeaa8f3e592e380c420e8910a717...	329.9	smart
7	7aeaa8f3e592e380c420e8910a717...	329.9	smart
8	7aeaa8f3e592e380c420e8910a717...	329.9	smart

Q4)

```
--q4 To identify seasonal sales patterns, Amazon India needs to focus on the most successful months.
--Determine the top 3 months with the highest total sales value, rounded to the nearest integer.
--Output: month, total_sales
SELECT
    EXTRACT(MONTH FROM O.order_purchase_timestamp) AS month,
    ROUND(SUM(I.price)) AS total_sales
FROM amazon_brazil.orders AS O
JOIN amazon_brazil.order_items AS I
  ON O.order_id = I.order_id
GROUP BY month
ORDER BY total_sales DESC
LIMIT 3;
```

Data Output Messages Notifications

	month numeric	total_sales double precision
1	5	1502589
2	8	1428658
3	7	1393539

Q5)

```sql
--q5Amazon India is interested in product categories with significant price variations.
--Find categories where the difference between the maximum and minimum product prices is greater than 500 BRL.
--Output: product_category_name, price_difference
SELECT  P.product_category_name,(max(O.price)-min(O.price)) as  price_difference
FROM amazon_brazil.product as P
join amazon_brazil.order_items as O
on P.product_id=O.product_id
group by P.product_category_name
```

Data Output Messages Notifications

	product_category_name character varying	price_difference double precision
1	climatizacao	1588.1
2	livros_importados	730.01
3	[null]	3977
4	artigos_de_natal	289.5
5	livros_tecnicos	374.98
6	ferramentas_jardim	3923.65
7	dvds_blu_ray	1411.1
8	fashion_roupa_feminina	179

Q6)

```sql
--q6 To enhance the customer experience,
--Amazon India wants to find which payment types have the most consistent transaction amounts
--Identify the payment types with the least variance in transaction amounts,
--sorting by the smallest standard deviation first.
--Output: payment_type, std_deviation
SELECT
    payment_type,
    ROUND(VARIANCE(payment_value)::numeric, 2) AS variance,
    ROUND(STDDEV(payment_value)::numeric, 2) AS std_deviation
FROM amazon_brazil.payments
GROUP BY payment_type
ORDER BY variance ASC
```

Data Output Messages Notifications

	payment_type character varying	variance numeric	std_deviation numeric
1	not_defined	0.00	0.00
2	voucher	13344.68	115.52
3	boleto	45616.87	213.58
4	credit_card	49336.99	222.12
5	debit_card	60414.40	245.79

Q7)

```sql
--q7Amazon India wants to identify products that may have incomplete name in order to fix it from their end.
--Retrieve the list of products where the product category name is missing or contains only a single character.
--Output: product_id, product_category_name
select product_id,product_category_name from amazon_brazil.product
where product_category_name is NULL or length(product_category_name)=1
```

Data Output Messages Notifications

	product_id [PK] character varying	product_category_name character varying
1	a41e356c76fab66334f36de622ecbd3a	[null]
2	d8dee61c2034d6d075997acef1870e...	[null]
3	56139431d72cd51f19eb9f7dae4d1617	[null]
4	46b48281eb6d663ced748f324108c7...	[null]
5	5fb61f482620cb672f5e586bb132eae9	[null]
6	e10758160da97891c2fdcbc35f0f031d	[null]
7	39e3b9b12cd0bf8ee681bbc1c130feb5	[null]
8	794de06c32a626a5692ff50e4985d36f	[null]

Analysis 2

Q1)

```sql
-- Analysis 2
--Segment order values into three ranges: orders less than 200 BRL, between 200 and 1000 BRL, and over 1000 BRL.
--Calculate the count of each payment type within these ranges and display the results in descending order of count
--Output: order_value_segment, payment_type, count
select payment_type,count(payment_type)as count,
case
when payment_value>=1000 then'High'
when payment_value between 200 and 1000 then 'Medium'
else 'Low'
end as  order_value_segment
from  amazon_brazil.payments
group by payment_type,order_value_segment
order by count desc
```

Data Output Messages Notifications

Showing ro

	payment_type character varying	count bigint	order_value_segment text
1	credit_card	60548	Low
2	boleto	16444	Low
3	credit_card	15303	Medium
4	voucher	5476	Low
5	boleto	3162	Medium
6	debit_card	1287	Low
7	credit_card	944	High
8	voucher	286	Medium

Q2)

```sql
--Amazon India wants to analyse the price range and average price for each product category.
--Calculate the minimum, maximum, and average price for each category,
--and list them in descending order by the average price.
--Output: product_category_name, min_price, max_price, avg_price

select product_category_name,max(price) as  max_price ,min(price) as min_price ,
avg(price) as avg_price
from amazon_brazil.product  as P
join amazon_brazil.order_items as U on
P.product_id =U.product_id
group by product_category_name
order by avg_price desc
```

Data Output Messages Notifications

	product_category_name character varying	max_price double precision	min_price double precision	avg_price double precision
1	pcs	6729	34.5	1098.34054187192
2	portateis_casa_forno_e_cafe	2899	10.19	624.2856578947369
3	eletrodomesticos_2	2350	13.9	476.12495798319287
4	agro_industria_e_comercio	2990	12.99	341.66104265402856
5	instrumentos_musicais	4399.87	4.9	281.6159999999985
6	eletroportateis	4799	6.5	280.77846833578695
7	portateis_cozinha_e_preparadores_de_alimentos	1099	17.42	264.5686666666667
8	telefonia_fixa	1790	6	225.69318181818173
9	construcao_ferramentas_seguranca	3099.9	8.9	208.99237113402086

Q3)

```
--Amazon India wants to identify the customers who have placed multiple orders over time.
--Find all customers with more than one order,
--and display their customer unique IDs along with the total number of orders they have placed.
--Output: customer_unique_id, total_orders
select customer_unique_id,count(order_id) as total_orders
from amazon_brazil.orders O
join  amazon_brazil.customer C
on O.customer_id=C.customer_id
group by customer_unique_id
having count(order_id)>1
order by total_orders desc
```

Data Output Messages Notifications

	customer_unique_id character varying	total_orders bigint
18	3d364a7768fae99678635c4370295d...	16
19	6af40347f5dd7bdd65437a35e1b2fa7b	16
20	f300b00a19af4d4f7bdf9f4524c4587a	16
21	75f15790b1852b42b1dbf645d98ffa1c	16
22	8d50f5eadf50201ccdcedfb9e2ac8455	15
23	7c396fd4830fd04220f754e42b4e5bff	14
24	ce2e0ace655301bc4a8cae4abbd8c0...	11
25	b11b7871c2b8be2d11fab954f58542f2	11

Q4)

```
--Amazon India wants to categorize customers into different types
--('New - order qty. = 1' ;  'Returning' -order qty. 2 to 4;  'Loyal' - order qty. >4)
--based on their purchase history.
--Use a temporary table to define these categories
--and join it with the customers table to update and display the customer types.
--Output: customer_unique_id, customer_type
```

```sql
With my_cte as(
Select customer_unique_id,count(*),
case
when count(*)=1 then 'New'
when count(*) between 2 and 4 then 'Returning'
else 'Loyal'
end as customer_type from amazon_brazil.orders O
join amazon_brazil.customer C on O.customer_id=C.customer_i
group by customer_unique_id)
select M.customer_unique_id,M.customer_type from my_cte M
join amazon_brazil.customer C
on M.customer_unique_id=C.customer_unique_id
order by M.customer_type
```

Data Output Messages Notifications

	customer_unique_id character varying	customer_type text
233	de34b16117594161a6a89c50b289d3...	Loyal
234	ca77025e7201e3b30c44b472ff346268	Loyal
235	ca77025e7201e3b30c44b472ff346268	Loyal
236	b643c19d91a65d73f94ba8e0376689...	Loyal
237	87aecbf5cf4d5e1f984e1d7a481c4f93	New
238	87b13b9164a289cec73d8daaa28cec...	New
239	87b162d0110779fd1d97a4e03f7407...	New
240	87b17699dbb8c555f40463875ccaea...	New
241	87b2e4a2df8e1635a1f89705abb6e94b	New

Q5)

```sql
--Amazon India wants to know which product categories generate the most revenue.
--Use joins between the tables to calculate the total revenue for each product category.
--Display the top 5 categories.
--Output: product_category_name, total_revenue

select product_category_name ,Sum((O.price-O.freight_value)) as total_revenue
from amazon_brazil.product P
join amazon_brazil.order_items O
on P.product_id=O.product_id
group by product_category_name
order by total_revenue desc
limit 5
```

Data Output Messages Notifications

	product_category_name character varying	total_revenue double precision
1	relogios_presentes	1102584.9300000004
2	beleza_saude	1075447.050000006
3	cama_mesa_banho	828447.2700000133
4	esporte_lazer	817570.540000009
5	informatica_acessorios	763556.8600000038

Analysis 3

Q1)

```sql
-- Analysis-3
 --Use a subquery to calculate total sales for each season (Spring, Summer, Autumn, Winter)
-- based on order purchase dates, and display the results.
-- Spring is in the months of March, April and May.
--Summer is from June to August and Autumn is between September and November
--and rest months are Winter.
--Output: season, total_sales
```

```sql
select season,total_sales from(
select
case
when extract(month from order_purchase_timestamp) in(3,4,5) then 'Spring'
when extract(month from order_purchase_timestamp) in(6,7,8) then 'Summer'
when extract(month from order_purchase_timestamp) in(9,10,11) then 'Autumn'
else 'Winter'
end as season,sum(OI.price-OI.freight_value) as total_sales
from  amazon_brazil.orders  as O join
 amazon_brazil.order_items as OI on
 O.order_id=OI.order_id
group by season) as st
```

Data Output Messages Notifications

	season text	total_sales double precision
1	Autumn	1971439.919999989
2	Spring	3538174.6199998935
3	Summer	3399866.369999941
4	Winter	2430253.2499999604

Q2)

```sql
--The inventory team is interested in identifying products that have sales volumes above the overall average.
--Write a query that uses a subquery to filter products with a total quantity sold above the average quantity.
--Output: product_id, total_quantity_sold
```

```sql
SELECT product_id, total_quantity_sold
FROM (
    SELECT
        P.product_id,
        COUNT(O.order_item_id) AS total_quantity_sold
    FROM amazon_brazil.order_items O
    JOIN amazon_brazil.product P
        ON O.product_id = P.product_id
    GROUP BY P.product_id
) AS product_totals
WHERE total_quantity_sold > (
    SELECT AVG(count_per_product)
    FROM (
        SELECT COUNT(OI.order_item_id) AS count_per_product
        FROM amazon_brazil.order_items OI
        GROUP BY OI.product_id
    ) AS avg_subquery
)
ORDER BY total_quantity_sold;
```

ata Output Messages Notifications

product_id [PK] character varying	total_quantity_sold bigint
c7df652246ed7b3300aaf46960c141e4	4
f89caa3ea4fb5f7b6e7fb2bb4ba61881	4
b2545dd62c61d45d55ebaa04d1cb1e...	4
c8d1837df49c5b4d91a388c8179515...	4

Q3)

```sql
--To understand seasonal sales patterns,
--the finance team is analysing the monthly revenue trends over the past year (year 2018).
--Run a query to calculate total revenue generated each month and identify periods of peak and low sales.
 --Output: month, total_revenue
SELECT
    TO_CHAR(DATE_TRUNC('month', order_purchase_timestamp), 'YYYY-MM') AS month,
    SUM(price - freight_value) AS total_revenue
FROM amazon_brazil.order_items oi
JOIN amazon_brazil.orders o
    ON oi.order_id = o.order_id
WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018
GROUP BY DATE_TRUNC('month', order_purchase_timestamp)
ORDER BY month;
```

Data Output Messages Notifications

	month text	total_revenue double precision
1	2018-01	792758.8300000103
2	2018-02	701448.4600000099
3	2018-03	811300.0600000006
4	2018-04	833597.460000002
5	2018-05	843253.5400000077
6	2018-06	707571.51
7	2018-07	732286.4099999993

Q4)

```sql
--A loyalty program is being designed  for Amazon India.
--Create a segmentation based on purchase frequency: 'Occasional' for customers with 1-2 orders,
--'Regular' for 3-5 orders,
--and 'Loyal' for more than 5 orders.
--Use a CTE to classify customers and their count .--Output: customer_type, count
with customer_orders as (
select count(*) as order_count ,  O.customer_id
from amazon_brazil.orders O
join amazon_brazil.customer C
on O.customer_id =C.customer_id
group by  O.customer_id)
select
case
when order_count between 1 and 2 then 'Occasional'
when order_count between 3 and 5 then 'Regular'
else 'Loyal'
end as customer_type,count(*) as customer_count
from customer_orders
group by customer_type
order by  customer_count
```

Data Output Messages Notifications

	customer_type text	customer_count bigint
1	Loyal	98
2	Regular	106
3	Occasional	98144

Q5)

```sql
--You are required to rank customers based on their average order value (avg_order_value)
--to find the top 20 customers.
--Output: customer_id, avg_order_value, and customer_rank

WITH customer_avg AS (
    SELECT
        O.customer_id,
        AVG(OI.price - OI.freight_value) AS avg_order_value
    FROM amazon_brazil.orders O
    JOIN amazon_brazil.order_items OI
        ON O.order_id = OI.order_id
    GROUP BY O.customer_id
)
SELECT
    customer_id,
    avg_order_value,
    DENSE_RANK() OVER (ORDER BY avg_order_value DESC) AS customer_rank
FROM customer_avg
ORDER BY customer_rank
LIMIT 20;
```

	customer_id character varying	avg_order_value double precision	customer_rank bigint
1	c6e2731c5b391845f6800c97401a4...	6540.69	1
2	f48d464a0baaea338cb25f816991a...	6535.79	2
3	3fd6777bbce08a352fddd04e4a7cc...	6271.34	3
4	df55c14d1476a9a3467f131269c24...	4647.66	4
5	24bbf5fd2f2e1b359ee7de94defc4a...	4615.66	5
6	3d979689f636322c62418b6346b1c...	4498.22	6
7	1afc82cd60e303ef09b4ef9837c950...	4286.42	7
8	35a413c7ca3c69756cb75867d6311...	4024.72	8
9	c6695e3b1e48680db36b487419fb0...	3982.89	9
10	e9b0d0eb3015ef1c9ce6cf5b9dcbee...	3954.49	10
11	3be2c536886b2ea4668eced3a80dd...	3917.26	11
12	31e83c01fce824d0ff786fcd48dad0...	3880.45	12
13	926b6a6fb8b6081e00b335edaf578...	3803.24	13
14	eb7a157e8da9c488cd4ddc48711f1...	3763.56	14
15	66657bf1753d82d0a76f2c4719ab8...	3663.7599999999998	15
16	19b32919fa1198aefc0773ee2e46e...	3607.41	16
17	addc91fdf9c2b3045497b57fc710e8...	3553.2	17
18	39d6658037b1b5a07d0a24d423f0b...	3495.53	18
19	e7c905bf4bb13543e8df947af4f3d9...	3273.52	19
20	a95f4bbcf95262b073e4afa481b59ff8	3043.98	20

Q6)

```sql
--Amazon wants to analyze sales growth trends for its key products over their lifecycle.
--Calculate monthly cumulative sales for each product from the date of its first sale.
--Output: product_id, sale_month, and total_sales

SELECT
    oi.product_id,
    TO_CHAR(DATE_TRUNC('month', o.order_purchase_timestamp), 'YYYY-MM') AS sale_month,
    SUM(oi.price - oi.freight_value) AS monthly_sales,
    SUM(SUM(oi.price - oi.freight_value))
        OVER (PARTITION BY oi.product_id ORDER BY DATE_TRUNC('month', o.order_purchase_timestamp)) AS total_sales
FROM amazon_brazil.order_items oi
JOIN amazon_brazil.orders o
    ON oi.order_id = o.order_id
GROUP BY oi.product_id, DATE_TRUNC('month', o.order_purchase_timestamp)
ORDER BY oi.product_id, sale_month;
```

Data Output Messages Notifications

Showing ro

	product_id character varying	sale_month text	monthly_sales double precision	total_sales double precision
	00066f42aeeb9f3007548bb9d3f33c38	2018-05	83.06	83.06
2	00088930e925c41fd95ebfe695fd2655	2017-12	115.97	115.97
3	0009406fd7479715e4bef61dd91f2462	2017-12	215.9	215.9
4	000b8f95fcb9e0096488278317764d19	2018-08	78.6	78.6
5	000d9be29b5207b54e86aa1b1ac548...	2018-04	179.73	179.73
6	0011c512eb256aa0dbbb544d8dffcf6e	2017-12	36.2	36.2
7	00126f27c813603687e6ce486d909d01	2017-09	468.27	468.27

Q7)

```sql
--To understand how different payment methods affect monthly sales growth,
--Amazon wants to compute the total sales for each payment method and
--calculate the month-over-month growth rate for the past year (year 2018).
--Write query to first calculate total monthly sales for each payment method,
--then compute the percentage change from the previous month.
--Output: payment_type, sale_month, monthly_total, monthly_change.
```

```sql
With monthly_sales as(
 select payment_type,sum(O.price-O.freight_value) as monthly_total,
 TO_CHAR(OI.order_purchase_timestamp,'YYYY-MM') AS sale_month
 from amazon_brazil.payments P
 join amazon_brazil.order_items O
 on P.order_id =O.order_id
 join amazon_brazil.orders OI
 on O.order_id=OI.order_id
 WHERE extract(year from OI.order_purchase_timestamp)=2018
 group by  payment_type,sale_month)
SELECT
    payment_type,
    sale_month,
    monthly_total,
    ROUND(
        (
            (monthly_total - LAG(monthly_total) OVER (PARTITION BY payment_type ORDER BY sale_month))
            / NULLIF(LAG(monthly_total) OVER (PARTITION BY payment_type ORDER BY sale_month), 0)
        )::NUMERIC, 2
    ) AS monthly_change
FROM monthly_sales
ORDER BY payment_type, sale_month;
```

	payment_type character varying	sale_month text	monthly_total double precision	monthly_change numeric
1	boleto	2018-01	138668.06000000006	[null]
2	boleto	2018-02	124726.94999999998	-0.10
3	boleto	2018-03	124488.46999999991	0.00
4	boleto	2018-04	132334.12999999998	0.06
5	boleto	2018-05	138048.90999999995	0.04
6	boleto	2018-06	99564.03999999996	-0.28
7	boleto	2018-07	128767.07999999991	0.29
8	boleto	2018-08	93042.13000000006	-0.28
9	credit_card	2018-01	639122.4600000002	[null]
10	credit_card	2018-02	568592.9199999985	-0.11
11	credit_card	2018-03	678228.7799999998	0.19
12	credit_card	2018-04	689809.2399999993	0.02
13	credit_card	2018-05	693612.020000001	0.01
14	credit_card	2018-06	585105.46	-0.16
15	credit_card	2018-07	573264.9600000008	-0.02
16	credit_card	2018-08	579338.679999999	0.01
17	debit_card	2018-01	7808.440000000004	[null]
18	debit_card	2018-02	4708.58	-0.40
19	debit_card	2018-03	5414.64	0.15
20	debit_card	2018-04	7209.910000000002	0.33
21	debit_card	2018-05	8023.8899999999985	0.11